

Prompting patterns that actually work

AI ENGINEERING 101
EPISODE 02

Five patterns that change what comes out of a model, each with a before and after you can copy. None of them are clever phrasing — they are structure. That is why they keep working as models improve.

01 Show, don't describe

Few-shot prompting. Don't describe the output you want — give two or three finished examples. A model is a pattern completer; hand it a pattern and it completes it. Highest return of anything on this sheet.

DESCRIBED

Keep it short.
Be professional.
Use bullet points.

SHOWN

Q: Order late?
A: Shipped Tue, arrives Thu.

Q: Wrong size?
A: Free return label sent.

Cost: examples ship on every call. Use the shortest pair that still carries the pattern — two good ones beat six mediocre ones.

02 Make it show its work

Chain of thought. Ask for reasoning before the answer and accuracy rises on anything with intermediate steps — a model generates one token at a time, so reasoning out loud gives it more tokens to think in.

NON-REASONING MODEL, NO COT

" $23 \times \$4.75 + 8 \times \3.20 ?"
→ one guess, often wrong

ADD: THINK STEP BY STEP

$23 \times 4.75 = 109.25$
 $8 \times 3.20 = 25.60$
→ \$134.85

Update: reasoning models already do this internally. Telling one to "think step by step" adds nothing and costs tokens. Use it on fast, cheap models; skip it on reasoning models.

03 Ask for a shape

Structured output. This is the one that turns a chatbot into software. Ask for JSON with named fields and the answer arrives as data your code reads — no parser, no regex, nothing to break when the model rephrases.

PROSE

"This looks like a billing issue and seems fairly urgent."

→ now write a parser

SCHEMA

```
{  
  "category": "billing",  
  "urgency": "high",  
  "needs_human": true  
}
```

Use your provider's built-in structured output or function calling rather than asking politely in the prompt and hoping.

04 Separate the rules from the task

The system prompt holds who the model is, what it must always do, what it must refuse, and what to say when it doesn't know. The user message holds only the task. Keeping them apart is what makes your rules survive a strange request.

```
# system
You triage support tickets.
If the ticket mentions money, set needs_human = true.
If unsure of the category, use "account".

# user
Charged twice this month, please help
```

A system prompt is not a security boundary. It is not hidden, it is not guaranteed, and it can be talked around. Never put a secret or an API key in one.

05 Contrast good with bad

Negative *instructions* are weak; negative *examples* are strong. “Don’t be verbose” works about as well as “don’t think about elephants” — you have just put the idea in the room.

A MOOD

Don’t be verbose.

CHECKABLE

Answer in at most two sentences.

But labelling a **bad example** next to a **good** one works well — the contrast does the teaching for you.

– The one that stopped working

Bribing and threatening: “I’ll tip you \$200”, “my career depends on this”, “take a deep breath”. The measured effect was always small and inconsistent, and on current models it is essentially gone.

Why it matters: tricks exploit a model’s weaknesses, so they expire every time the model improves. Patterns work with how the model functions, so they last. Everything else on this sheet is a pattern.

How to actually improve a prompt

01 Change one thing at a time.

Edit the examples and the instructions together and you learn nothing about either.

02 Keep a fixed set of test inputs.

Ten or twenty real cases in a file. Same set, every version — otherwise you are comparing a new prompt on easy inputs to an old one on hard inputs.

03 Read the failures, not the successes.

A prompt that works on eight cases out of ten is defined by the two.

Past a handful of test cases you have outgrown eyeballing it. That is evaluation — the fourth pillar — and it is where the playlist ends up.

